Introduction To Tek271 Memoizer

A Java open source memoization library to cache the results of slow methods using annotations and a pluggable cache interface.

By: **Abdul Habra**
Email: ahabra@yahoo.com
URL: www.tek271.com
Version:1.1
Date: 2009.06.27

This software is open source, free, and uses LGPL license.

Version	**Download** *(source, Jar, JavaDocs)*	**Docs**
1.1	tek271.memoize.2009.06.27.zip	API
1.01	tek271.memoize.2007.03.05.zip	
1.0	tek271.memoize.2007.03.01.zip	

---

1. Background

If a function produces the same output given the same inputs, and if this function is slow, it makes sense for the function to cache its outputs to avoid repeated evaluations. This caching behavior is called *Memoization* (yes there is no **r**).

For example consider a method that reads some authorization string from database for a given login:

```
// This is not ideal code, it is meant only to show a concept
public String readAuthorizationFromDb(String loginId) {
  Connection con= getDbConnection();   // perhaps pooled
  String sql= "select authorization from security where login_id='" + loginId + "'";
  String auth= readFromDb(con, sql);   // assume we have such a method
  con.close();
  return auth;
}
```

If this function is called many times, with the same loginId, a programmer may consider this kind of optimization:

```
Map cache= new HashMap();

// This is not ideal code, it is meant only to show a concept
public String readAuthorizationFromDb(String loginId) {
  if (cache.containsKey(loginId)) {
    return (String) cache.get(loginId);
  }

  Connection con= getDbConnection();   // perhaps pooled
  String sql= "select authorization from security where login_id='" + loginId + "'";
  String auth= readFromDb(con, sql);   // assume we have such a method
  con.close();

  cache.put(loginId, auth);
  return auth;
}
```

This solution could work for a simple program, however it suffers from several problems such as:

1. There is no control on the size of the map. If the method is called many times with different `loginIds`, the size of map can increase and run out of memory.
2. There is no control on expiring items in the cache. Once a `loginId` is put in the map it stays there, if the database contents change, there is no way to update the content of the map besides restarting the program.
3. Having cache handling code in every method that needs caching is annoying and error prone.

One possible solution is to use Aspects to cache this kind of method. However, the solution provided here uses standard Java, and is lighter in weight. The following is the method's code using Tek271 Memoizer:

```
// This is not ideal code, it is meant only to show a concept
@Remember
public String readAuthorizationFromDb(String loginId) {
  Connection con= getDbConnection();   // perhaps pooled
  String sql= "select authorization from security where login_id='" + loginId + "'";
  String auth= readFromDb(con, sql);   // assume we have such a method
  con.close();
  return auth;
}
```

As you can see, the only needed addition is to add the `@Remember` annotation before the method's declaration.

2. Required Setup

1. You need JDK 5 or higher (to support annotations).
2. The cglib jar file, currently it is `cglib-nodep-2.2.jar`. Available from http://cglib.sourceforge.net. It is also included in this project's download.
3. The jar file for Tek271 Memoizer, currently `tek271.memoize-1.1.jar`.

3. Example

```
import com.tek271.memoize.Remember;

public class ExpensiveCalcs {
  @Remember
  public String slowMethod(String param1) {
    System.out.println("inside slowMethod(" + param1 + ")");
    return param1 + param1;
  }
}
```

Now, to use the slowMethod():

```
import com.tek271.memoize.RememberFactory;
...

ExpensiveCalcs ec= RememberFactory.createProxy(ExpensiveCalcs.class);
String s= ec.slowMethod("hello");
```

Notice how you use `RememberFactory.createProxy()` to create instances of `ExpensiveCalcs` instead of using a constructor.

An alternative approach is also available (added in version 1.1)

```
ExpensiveCalcs ec= new ExpensiveCalcs(); // can be created through dependency injection
ExpensiveCalcs decorated= RememberFactory.decorate(ec);
String s= decorated.slowMethod("hello");
```

This approach can be used when you have an existing object (rather than a class). For example, if the object you have was created using some dependency injection framework like *Spring*.

4. `@Remember`'s Attributes

The `@Remember` annotation has several attributes that can be used to modify its default behavior.

maxSize
The maximum number of method results to cache. The default is 128. For example, to change the maxSize to 1000:

```
@Remember(maxSize=1000)
```

timeToLive
The period of time after which, cached return values of the method will expire. The default is 2 minutes. This attribute works in conjunction with the `timeUnit` attribute.

timeUnit
The unit of time for the timeToLive attribute. The default is Minute. This is an enumerations with possible values of {MILLI, SECOND, MINUTE, HOUR}. For example, to change the timeToLive to 4 hours:

```
@Remember(timeToLive=4, timeUnit=TimeUnitEnum.HOUR)
```

excludedParametersIndex
Method parameters that should NOT be used as part of the cache's key. The excluded parameters are indicated in an `int` array. The java reflection API does not provide access to method's parameters names, hence we use index. The index of the first parameter is zero. For example assume that you have a method with three parameters:

```
@Remember
String readUsersAddress(Connection con, String city, String state) {
  ...
}
```

To memoize this method, you realize that the connection parameter should not impact the return value of the method:

```
@Remember(excludedParametersIndex={0})
String readUsersAddress(Connection con, String city, String state) {
  ...
}
```

Important reminder: method parameters which are used in the cahce's key must support correct `equals()` and `hashCode()` methods.

5. Pluggable Caching

The Tek271 Memoizer stores its cached values using a light-weight implementation of a cache store using `java.util.LinkedHashMap` class. However, you can use any other caching software by implementing the provided `ICacheFactory` and `ICache` interfaces and using this factory method:

```
RememberFactory.createProxy(Class targetClass, ICacheFactory cacheFactory)
```

An implementation that uses Ehcache is available.

6. References

1. Memoization in Java Using Dynamic Proxy Classes by Tom White. www.onjava.com/pub/a/onjava/2003/08/20/memoization.html. This solution however requires creating interfaces for each class that have memoized methods.

2. Create Proxies Dynamically Using CGLIB Library by Jason Zhicheng Li. www.ociweb.com/jnb/jnbNov2005.html
3. Explore the Dynamic Proxy API by Jeremy Blosser. www.javaworld.com/javaworld/jw-11-2000/jw-1110-proxy.html
4. The Java Docs for JSE 5.0 and cglib.

Changes

Version 1.1, 2009.06.27

1. Updated dependency to latest cglib jar (version 2.2)
2. Added RememberFactory.clearCache() methods to ease unit testing.
3. Added an optimization proposed by *Christian Semrau*.
4. Added Generics support to RememberFactory.createProxy()
5. Added RememberFactory.decorate() to decorate a given object (rather than class). This feature was requested by *Patrick McMichael*.

Version 1.01, 2007.03.05

Some JavaDocs fixes and spelling mistakes.

Version 1.0, 2007.03.01

First public release

Page Last Updated 2009.06.27